O&M Cane Training

Native (Android) Build Log & Code Map

Capacitor 8 wrap · Phase 1 · Generated for project handoff

This document tracks the work done to wrap the teacher app as a native Android app, records every change made along the way, and maps what each part of the codebase does. It is written so a future session (or another developer) can pick up without re-deriving context. It is a working engineering log, not a spec.

1. Where the build stands right now

VERIFIED WORKING

The app builds and launches as a native Android app on the emulator, and **data survives a cold restart** — meaning native storage (Capacitor Preferences) is genuinely persisting data, not silently falling back to WebView storage. This was the single riskiest, least-documented part of the no-bundler approach, and it is now retired.

Step	Status
Homebrew, Node 26, Android Studio, emulator (API 37, arm64-v8a)	DONE
Files placed in `~/Desktop/om-app`; `www/` created and populated	DONE
`npm install` → Capacitor 8.4.0 + 4 plugins	DONE
`npx cap add android` → native project created, 4 plugins detected	DONE
JDK resolution — pointed terminal at Android Studio's bundled JDK 21	DONE
`npx cap run android` → builds & launches on emulator	DONE
Cold-restart storage test — child persists after kill + relaunch	PASSED
On-device checks: CSV share sheet, DOB picker, delete confirm	VERIFY NEXT
Run on a real phone (USB)	OPTIONAL

2. The Java fix (what went wrong and why)

The first `npx cap run android` failed with `Unable to locate a Java Runtime`. This is expected on a fresh Mac: Gradle needs a JDK, but the command line doesn't know where one is. The correct JDK (version 21) ships *inside* Android Studio — Java should not be installed separately from java.com.

The fix pointed the terminal at the bundled JDK by setting `JAVA_HOME` in `~/.zshrc`:

```
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
export PATH=$JAVA_HOME/bin:$PATH
```

After `source ~/.zshrc`, `java -version` reported **openjdk 21.0.10**, and the build proceeded. This setting persists across terminal sessions, so it does not need redoing.

3. What the native wrap added (vs the plain web app)

The wrap was deliberately **additive**: the web app still works exactly as before (double-click `index.html` or run a local server), and the no-coder `activities.js` edit-refresh workflow is untouched. The native layer sits underneath.

New files created for the wrap

File	Purpose
`capacitor.config.json`	App identity: appId `org.omcane.trainer`, appName "O&M Cane Training", serves from `www/`, splash + background `#f7f4ed` (matches the app's warm paper).
`package.json`	Declares Capacitor 8 core + CLI + the 4 plugins (Preferences, Filesystem, Share, SplashScreen) + the Android platform package. `npm install` reads this.
`BUILD-ANDROID.md`	The full click-by-click Mac setup guide, with a "Check" after every step and a troubleshooting section.
`android/ (folder)`	The actual native Android project, generated by `npx cap add android`. Not hand-edited.
`www/ (folder)`	What Capacitor bundles. Holds copies of `index.html` + `activities.js`. The build step is copying source into here.

Code changed inside index.html

The substantive change was the **storage layer**, rewritten so the same code runs on both web and native. Everything below is in `index.html`.

4. Code map — what each part does

The Store seam (the heart of the wrap)

The problem it solves: native storage (Capacitor Preferences) is *asynchronous* — every read returns a Promise. But all the screen-drawing code reads storage *synchronously* while building HTML. Rewriting every render function to `await` reads would be a huge, fragile change.

The solution — a cache-backed hybrid:

- `Store.init()` — async, runs **once** at boot. Loads the entire storage backend into a synchronous in-memory cache. Boot waits for this before drawing the first screen.
- **Reads** (`getJSON`, `getString`, `_keys`) — stay **synchronous**, served from the cache. This is why none of the render functions had to change.
- **Writes** (`setJSON`, `setString`, `_remove`) — **async**. They update the cache immediately (so the next read is correct), write through to the backend, then **read back to verify**. The Promise resolves `true` only when the backend confirms the write.

Why read-back-verify: inside a native WebView a quota or OS error can fail *silently* — the worst outcome for a children's data tool. A failed write now surfaces to the teacher (who can export a CSV) instead of vanishing.

To change storage backends later: only the four `_backend*` methods and the detection in `init()` need editing. The cache, verify logic, and public API stay put.

Backend detection

At boot, `Store.init()` checks `window.Capacitor.isNativePlatform()` and whether the Preferences plugin is present. If both are true → native Preferences. Otherwise → browser localStorage. The cold-restart test confirmed the native branch is the one running on the device.

Data helpers built on the seam

Function group	What it does
`loadRecords / saveRecord`	Session results per activity, keyed `rec_<activityId>`. `saveRecord` returns true only on confirmed write.
`loadProfiles / upsertProfile / profileById`	Child profiles (name, DOB, height, weight, dominant hand, optional low-res photo), keyed `profiles`. Age is derived from DOB, never stored.
`getActiveProfile / setActiveProfileId`	Which child the next saved record attaches to.
`deleteRecord / deleteProfile / clearAllData`	Three deletion tiers. `deleteProfile` also sweeps every record linked to that child. `clearAllData` wipes everything but keeps the welcome-seen flag.
`recordCountForProfile`	

	Counts saved results for one child across all activities (shown before deleting).
`ageFromDOB / downscaleImage`	Derive age live; shrink a chosen photo to ~160px JPEG before storing.

CSV export — made backend-aware

`exportCSV()` now branches. On **native**, the browser's `<a download>` trick doesn't reliably save inside a WebView, so it writes the CSV to the app's cache directory via `@capacitor/filesystem`, then hands it to the OS **share sheet** via `@capacitor/share` (save to Files/Drive/email). On **web**, the original anchor-download path is unchanged. The CSV includes demographics (DOB, derived age, height, weight, dominant hand) and uses a UTF-8 BOM so Excel reads Indian-language names correctly. This export is currently the only backup of on-device data.

Boot sequence

```
boot() → await Store.init() → showWelcome('fwd')
```

Wrapped in try/catch: if init fails, the app still paints the Welcome screen (which reads nothing) and shows a toast, so the user never sees a blank screen. The app lands on Welcome every launch by design.

The three screens + Manage-data (unchanged by the wrap)

- **Welcome** → app name, tagline, "Get started".
- **Hub** → child capture form (collapses to a chip once saved) + Teach / Export / Manage-data actions.
- **Activities list** → flat list of all activities, category colour kept as a left spine.
- **Run screen** → child chip, the record form first, collapsible SOP with audio/video slots, past results last.
- **Manage-data** → off the Hub; per-child delete + a double-confirmed "clear all" danger zone; nudges toward CSV export first.

5. What to do next

1. **Finish the on-device verification list** (in the running emulator): Export records opens the share sheet; the Date-of-birth picker opens without being clipped by the sliding form; deleting a result shows and completes its confirm dialog.
2. **Run on a real phone** (optional but recommended for the pilot): enable USB debugging, plug in, `npx cap run android`, pick the physical device, and repeat the cold-restart storage test — that's the environment that actually matters.
3. **Then features, in order:** demo video into the existing `videoFile` slot (blocked only on clips); per-child consent field; session-video capture + upload (a new uploader seam mirroring Store, India-region cloud bucket); admin access.
4. **Cosmetic / release:** app icon + splash art, then eventually a signed build for the Play Store and an iOS target.

6. Watch list (known gotchas)

WATCH **Emulator version gap.** The emulator is API 37, arm64-v8a; the app is built against compileSdk 36. It should run (the app supports anything ≥ 24), but if the build or storage behaves oddly later, rule this mismatch out first.

WATCH **invalid source release: 21.** If this appears, Android Studio isn't using its bundled JDK. In Android Studio: Settings → Build, Execution, Deployment → Build Tools → Gradle → Gradle JDK → pick the bundled jbr-21.

WATCH **AGP / Gradle version complaint.** Open the project in Android Studio (`npx cap open android`), let the AGP Upgrade Assistant run if offered, target AGP 8.13.

7. The day-to-day workflow from here

Content team (web, unchanged): edit `activities.js`, save, refresh the browser. No build step, no terminal.

Shipping a change into the Android app (only when you want a new build):

```
cp index.html activities.js www/
npx cap sync android
npx cap run android
```

That copy + sync is the entire "build step" — two extra commands, only when producing a native build, and it never sits between the content team and their edit-refresh loop.